

Curso de Datos Móviles y Machine Learning para la Estimación de la Pobreza

Banco Mundial - USFQ

Instructores: Pablo Astudillo / Ignacio Garrón

15 de junio de 2026

Quiénes somos



Pablo Astudillo

Teoría

- Profesor de Economía, Colegio de Economía, USFQ.
- Director del **USFQ Data Hub**: ciencias sociales computacionales y humanidades digitales.
- PhD en Geografía Económica & Economía de la Complejidad, University of Oxford.



Ignacio Garrón

Práctica

- Professor de Econometría, Dept. de Estadística, U. Carlos III de Madrid.
- PhD en Economía, Universitat de Barcelona.
- Master en Economía, Barcelona School of Economics.

Antes de empezar: ¿quiénes son ustedes?

Participemos en [Menti](#) — [menti.com](#), código: **5778 9888**



Agenda del curso

Módulo 1. Pipeline de ML, prevención de *leakage* y preparación de datos reproducible en R.

Módulo 2. Modelos basados en árboles (Random Forest y Gradient Boosting): entrenamiento, validación cruzada y métricas.

Módulo 3. Modelos alternativos (SVM, k-NN, Naive Bayes): supuestos, selección y comparación empírica.

Módulo 4. Desbalance, calibración, interpretabilidad (importancia, PDP, SHAP) y exploración con PCA y k-means.

Bloque	Fecha	Inicio	Fin	Tipo
Modulo 1	15/6/26	14:00	15:30	Teoría
		15:30	17:00	Lab
Módulo 2	16/6/26	14:00	15:30	Teoría
		15:30	17:00	Lab
Módulo 3	17/6/26	14:00	15:30	Teoría
		15:30	17:00	Lab
Módulo 4	18/6/26	14:00	15:30	Teoría
		15:30	17:00	Lab
Ejercicio integrador	19/6/26	10:00	13:00	Lab/Cierre

Módulo 1: Agenda

1. Introducción a Machine Learning (*con ejercicio T-P-E*)
2. Las cinco tribus del aprendizaje
3. El ciclo de un proyecto de ML
4. Partir los datos y fuga de información (*leakage*)
5. Preparación de variables: imputación, codificación, escalamiento
6. Pipelines reproducibles (`tidymodels`)
7. Línea base: regresión lineal y logística
8. Reproducibilidad y *model cards*
9. Síntesis y Q&A

Al finalizar este módulo el participante será capaz de:

1. Comprender las ideas básicas de ML y sus variantes.
2. Ubicar el modelado dentro del ciclo completo de un proyecto de ML.
3. Partir los datos honestamente y detectar/prevenir fugas de información.
4. Preparar los datos (imputar, codificar, escalar) sin filtrar información.
5. Encapsular el preprocesamiento en un pipeline reproducible.
6. Usar la regresión lineal y logística como línea base obligatoria.
7. Documentar un modelo con criterios de *model card*.

Introducción a Machine Learning

¿Qué es *Machine Learning*?

Tom Mitchell (1998)

*“Se dice que un programa **aprende** de la experiencia **E** con respecto a una tarea **T** y una medida de desempeño **P**, si su desempeño en **T**, medido por **P**, mejora con la experiencia **E**.”*

Aplicado al filtro de *spam*:

- **T**: clasificar correos como spam o no-spam.
- **E**: correos previamente marcados (o no) por el usuario.
- **P**: proporción de correos clasificados correctamente.

Su turno: definan **T**, **P** y **E**

Instrucciones (en grupos de 3 · 5 minutos)

Elijan un problema de su trabajo y descríbanlo como la terna **T-P-E**. Si no logran especificar las tres, quizá todavía *no* sea un problema de ML.

Recordatorio:

- **T** (Tarea): qué se quiere predecir o decidir.
- **E** (Experiencia): qué datos o ejemplos alimentan el aprendizaje.
- **P** (Desempeño): cómo se mide si lo hace bien.

Ideas para elegir: clasificación de ocupación, imputación de ingreso faltante, predicción de no-respuesta.

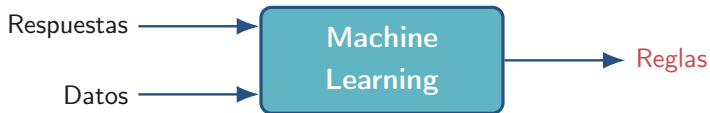
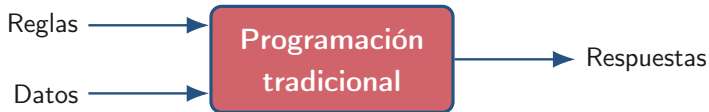
Ejemplo resuelto: focalización de pobreza:

Terna T-P-E

- T** clasificar un hogar como pobre / no pobre.
- E** hogares encuestados con su estatus ya medido.
- P** % de hogares clasificados correctamente.

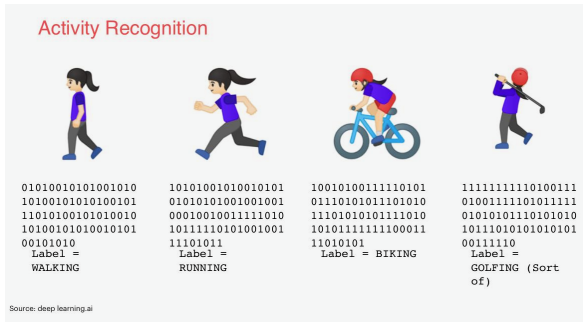
Ahora describan el suyo: una frase por cada letra.

Programación tradicional vs. Machine Learning



El cambio de paradigma: en lugar de escribir reglas, las *descubrimos* a partir de ejemplos.

Cuando las reglas no bastan: reconocimiento de actividad



Empezamos con `if (speed < 4) WALKING`; agregamos correr; agregamos pedalear; luego **golf**... y las reglas explotan.

ML invierte el problema: damos ejemplos etiquetados (*datos sensoriales* → *actividad*) y el modelo *infiere* las reglas.

Analogía del pastel de Yann LeCun:

- **Supervisado:** las respuestas correctas son dadas.
- **No supervisado:** sin respuestas; buscamos estructura.
- **Por refuerzo:** *(la cereza)* aprender de recompensas, ensayo y error.
- **Auto-supervisado:** el modelo se inventa las etiquetas a partir de los datos.

Machine learning flavors

Yann LeCun Cake Analogy

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning 🍒
- Self-Supervised Learning



<https://www.youtube.com/watch?v=Dart2T4q4Qo>

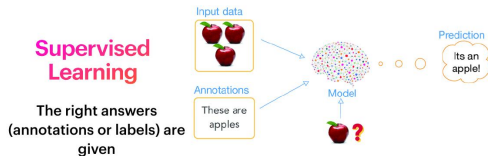
Aprendizaje supervisado: las respuestas se dan

Idea: se entrega al modelo pares (x_i, y_i) y aprende a predecir y a partir de x .

Dos sabores:

- **Regresión:** y continuo (ingreso, gasto).
- **Clasificación:** y discreto (pobre / no pobre).

En el curso: dominará este enfoque (pobreza, ingreso, focalización).



Source: https://www.researchgate.net/publication/329533120_Background-Augmentation-Generative-Adversarial-Networks-BAGANs-Effective-Data-Generation-Based-on-GAN-Augmented-3D-Synthesizing/figures?tool=figshare

Aprendizaje no supervisado: sin etiquetas

Idea: solo se tienen los x_i ; el modelo descubre *estructura* en los datos.

Tareas típicas:

- **Clustering** (segmentación de hogares).
- **Reducción de dimensión** (PCA, UMAP).
- **Detección de anomalías.**



Source: https://www.researchgate.net/publication/329533120_Background_Augmentation_Generative_Adversarial_Networks_BAGANs_Effective_Data_Generation_Based_on_GAN-Augmented_3D_Synthesizing/figures?lo=1

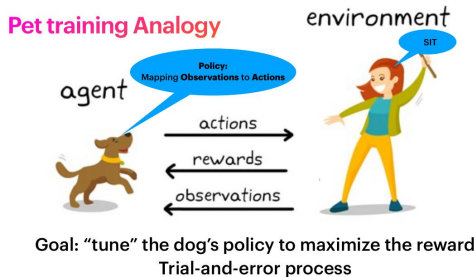
Útil cuando la etiqueta es cara o imposible de obtener.

Aprendizaje por refuerzo: ensayo, error, recompensa

Elementos:

- **Agente** que toma decisiones.
- **Entorno** que responde.
- **Política**: observación $\rightarrow$ acción.
- **Recompensa**: señal escalar a maximizar.

Ejemplos: robótica, juegos, conducción autónoma, ruteo de transporte público.



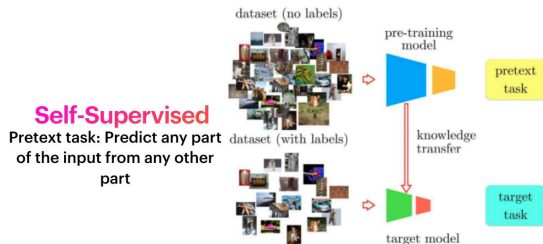
<https://www.mathworks.com/discovery/reinforcement-learning.html>

Aprendizaje auto-supervisado (SSL)

Tarea pretexto: predecir una parte del dato a partir de otra (sin etiquetas humanas).

Tarea objetivo: se aprovechan las representaciones aprendidas para una tarea con pocas etiquetas (*transfer learning*).

Ejemplo emblemático: los LLM (ChatGPT, Claude) predicen la siguiente palabra durante el pre-entrenamiento.



<https://neptune.ai/blog/self-supervised-learning>

Las cinco tribus del aprendizaje

Las cinco tribus del aprendizaje (+ una) (Domingos, 2015)

	Analogistas	Simbolistas	Conexionistas	Causales +uno	Bayesianos	Evolucionistas
El aprendizaje es...	<i>... reconocer similitud</i>	<i>... lógica</i>	<i>... el cerebro</i>	<i>... contrafactual</i>	<i>... actualizar creencias</i>	<i>... selección natural</i>
Representación	Vectores de soporte	Lógica y reglas	Redes neuronales	Resultados potenciales	Modelos gráficos	Programas genéticos
Evaluación	Margen	Exactitud	Error cuadrático	Ortogonalidad	Probabilidad posterior	Aptitud
Optimización	Optimización con restricciones	Deducción inversa	Descenso de gradiente	Cross-fitting	Inferencia probabilística	Búsqueda genética
Modelos de ejemplo	kNN, Máquinas de Vectores de Soporte	Árboles de decisión	Aprendizaje profundo	Double ML, bosques causales	Naive Bayes / Markov	Prog. evolutiva
Aplicación	Política industrial (densidad de relación)	Transferencias monetarias condicionadas (TMC)	Predecir riesgo de deserción escolar	Microfinanzas	Monitoreo de brotes de enfermedades	Rutas de buses escolares
Datos	Ubicación de empresas/industrias, clientes y proveedores	Encuestas de hogares	Datos a nivel de estudiante (asistencia, notas, distancia a la escuela)	Encuestas de hogares	Datos ruidosos de múltiples fuentes (hospital, telefonía móvil, censo antiguo)	Simulación y mapas

Domingos, Pedro. (2015). *The master algorithm: How the quest for the ultimate learning machine will remake our world*. Basic Books.

El ciclo de un proyecto de ML

Un proyecto de ML tiene siete etapas

Entrenar el modelo es **solo una** de ellas.



El **grueso del esfuerzo** está en *encuadrar* el problema y *preparar* los datos no en el algoritmo.

Partir los datos

Antes que nada: parte tus datos

La analogía del examen

Entrenar un modelo y evaluarlo con los **mismos** datos es como rendir un examen con las respuestas a la vista: la nota se ve alta, pero *no mide nada útil*.

La partición típica:

Train · 60–70 % aquí el modelo APRENDE.

Validación · 15–20 % ajustar hiperparámetros. DECIDE

Test · 15–20 % apartado hasta el final. JUZGA

El conjunto de prueba es **sagrado**: no se toca hasta el cierre.

Fuga de información (*data leakage*)

Información que **no estaría disponible en producción** se filtra al entrenamiento o a la evaluación, inflando artificialmente el desempeño estimado.

Es la causa más común de modelos “demasiado buenos para ser ciertos”.

Síntoma típico: un AUC sospechosamente alto ($> 0,99$) sin justificación.

Las tres fugas más comunes

Fuga 1: Transformar antes de partir

Calcular la media o escalar usando *todo* el dataset y luego partir.

Solución: parte primero; estima cada transformación (media, mín-max, otra) solo en el entrenamiento y aplícala igual al test, sin recalcular.

Fuga 2: Variables que vienen del futuro

Incluir como predictor algo que solo se conoce *después* de la decisión.

Solución: antes de añadir una variable, pregunta: ¿la tendré al momento de predecir?

Fuga 3: Partir por la unidad equivocada

Filas por persona, pero la unidad es el hogar; lo aleatorio mezcla a la misma familia.

Solución: parte por hogar, no por persona: todos los miembros de un hogar van juntos al train o juntos al test, nunca repartidos.

Regla de oro: *train* enseña, *test* evalúa, y nunca se cruzan.

Preparación de variables

Qué hacer con cada tipo de variable

Catégoricas (sin orden) *One-hot encoding*: una columna binaria por categoría.

INEC: provincia, sexo, estado civil

Catégoricas (con orden) *Ordinal encoding*: un número que respeta el orden.

INEC: nivel educativo, satisfacción

Valores faltantes imputar (mediana / moda) y, opcional, añadir *bandera* de faltante.

INEC: ingreso reportado, acceso a internet

Numéricas con escalas distintas estandarizar (*Z-score*) para modelos sensibles a la escala.

INEC: ingreso, edad, número de personas

Numéricas para árboles **sin escalar**: no son sensibles a la escala.

Random Forest, Gradient Boosting

One-hot encoding: en la práctica

Convertir una categórica en columnas binarias que el modelo pueda procesar.

Antes	
Hogar	Provincia
H001	Pichincha
H002	Guayas
H003	Azuay

Después			
Hogar	Pichincha	Guayas	Azuay
H001	1	0	0
H002	0	1	0
H003	0	0	1

Cuidado

Si la categoría tiene muchísimos valores (> 50), *one-hot* crea demasiadas columnas y conviene otra estrategia. **Lo veremos en el Día 3.**

Pipelines reproducibles

Cinco paquetes que veremos una y otra vez en todos los módulos.

<code>rsample</code>	partir en train/test y crear folds de CV.	<code>initial_split()</code> , <code>vfold_cv()</code>
<code>recipes</code>	definir las transformaciones de variables.	<code>recipe()</code> , <code>step_*</code> ()
<code>parsnip</code>	declarar un modelo de forma uniforme.	<code>logistic_reg()</code> , <code>rand_forest()</code>
<code>workflows</code>	empaquetar receta + modelo en un objeto.	<code>workflow()</code> , <code>add_recipe()</code>
<code>yardstick</code>	calcular métricas de evaluación.	<code>metric_set()</code> , <code>accuracy()</code>

La filosofía: receta → modelo → workflow → fit → predict → métricas.

Una sola gramática para todo el curso.

Línea base: regresión lineal y logística

Regresión lineal: la línea base más simple

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$h_{\theta}(x)$ la predicción del modelo para la entrada x .

θ_0 intercepto: valor predicho cuando $x = 0$.

θ_1 pendiente: cuánto cambia la predicción si x sube una unidad.

Ejemplo INEC

$h(x) = 180 + 220 \cdot x \Rightarrow$ un hogar con 2 personas trabajando: $h(2) = 620$ dólares.

Misma combinación lineal, pero la pasamos por una *sigmoide* para obtener una **probabilidad**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$$

Lectura del resultado

$h(x) = 0,78 \rightarrow 78\%$ de probabilidad de que ese hogar responda por internet.

Decisión por defecto: si $h(x) \geq 0,5$ predecimos 1; si no, 0.

El umbral 0.5 se puede mover según el costo de cada error. *Lo veremos en el Día 4.*

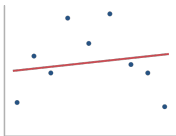
Tres regímenes de un modelo

Tu modelo siempre cae en uno de los tres. El objetivo es estar en el medio.

Sub-ajuste

Modelo demasiado simple.

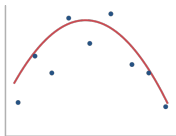
Error **alto** en train y en validación.



Justo

Captura la señal sin memorizar.

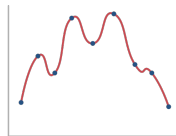
Error *similar* en train y validación.



Sobre-ajuste

Memoriza el ruido del train.

Error *muy bajo* en train, **alto** en validación.



La **regularización** (parámetro λ) es el freno que evita el sobre-ajuste.

Regularización (el freno técnico): Ridge, Lasso y Elastic Net

Idea: al error de ajuste le sumamos una **penalización** por coeficientes grandes.

$$\text{Costo}(\theta) = \underbrace{\text{MSE}}_{\text{ajuste a los datos}} + \lambda \underbrace{\text{Penalización}(\theta)}_{\text{castigo a coeficientes grandes}}$$

Ridge (L2) : $\text{MSE} + \lambda \sum_{j=1}^p \theta_j^2$ suaviza todos los coeficientes

Lasso (L1) : $\text{MSE} + \lambda \sum_{j=1}^p |\theta_j|$ puede volver coeficientes a 0

Elastic Net : $\text{MSE} + \lambda [\alpha \sum_j |\theta_j| + (1 - \alpha) \sum_j \theta_j^2]$ mezcla de ambas

λ fija la fuerza (se elige por validación cruzada). α mezcla L1/L2: $\alpha = 1$ es Lasso, $\alpha = 0$ es Ridge. El intercepto θ_0 no se penaliza.

Reproducibilidad y *model cards*

Lista mínima de reproducibilidad

Que cualquier persona del equipo pueda **repetir** tu resultado, paso a paso.

Fija el azar usa una *semilla* fija para que las particiones y los modelos con aleatoriedad den siempre el mismo resultado.

Versiona el código guarda el historial de cambios (control de versiones) con hitos claros.

Versiona los datos registra la fecha de corte o una huella del archivo; si los datos cambian, se nota.

Congela las versiones anota qué programas y paquetes usaste y en qué versión exacta.

Documenta el flujo una página que explique cómo correr todo desde cero.

La reproducibilidad no es opcional: es parte del rigor en estadística.

Una hoja que acompaña a cada modelo desplegado y hace el trabajo **auditable**.

Propósito para qué decisión y quién lo usa.

Datos fuente, periodo, tamaño, exclusiones, sesgos conocidos.

Métricas resultados en test, desglosadas por subgrupos relevantes.

Limitaciones casos donde **NO** debe usarse.

Consideraciones éticas riesgos, manejo de variables sensibles.

Mantenimiento responsable, cadencia de reentrenamiento, monitoreo.

Síntesis y Q&A

Cinco ideas para llevarte a casa

1. Antes de algoritmos, escribe T, P y E. Si no puedes, el problema no está listo.
2. El esfuerzo está en encuadrar el problema y preparar los datos, no en modelar.
3. Separa primero, transforma después. El test no se toca hasta el cierre.
4. Empaqueta el preprocesamiento en un recipe dentro de un workflow: es tu defensa contra la fuga.
5. La regresión lineal y la logística son la línea base obligatoria.

Módulo 2: árboles, bosques y boosting

¿Preguntas?

-  Agresti, A. (2015). *Foundations of Linear and Generalized Linear Models*. Wiley.
-  Buelens, B., Burger, J., & van den Brakel, J. (2014). Predictive inference for non-probability samples. *Statistics Netherlands DP*.
-  Domingos, P. (2015). *The Master Algorithm: How the Quest for the Ultimate Learning Machine Will Remake Our World*. Basic Books.
-  Elbers, C., Lanjouw, J.O., & Lanjouw, P. (2003). Micro-level estimation of poverty and inequality. *Econometrica* 71(1).
-  Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for GLMs via coordinate descent. *JSS* 33(1).

-  Gebru, T. et al. (2021). Datasheets for datasets. *CACM* 64(12).
-  Gneiting, T., & Raftery, A.E. (2007). Strictly proper scoring rules. *JASA* 102(477).
-  Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.
-  James, G. et al. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.
-  Jean, N. et al. (2016). Combining satellite imagery and ML to predict poverty. *Science* 353(6301).
-  Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in ML-based science. *Patterns* 4(9).

-  Kuhn, M., & Johnson, K. (2019). *Feature Engineering and Selection*. CRC Press.
-  LeCun, Y. (2016). Predictive Learning. NeurIPS Keynote.
-  Lumley, T. (2010). *Complex Surveys*. Wiley.
-  McBride, L., & Nichols, A. (2018). Retooling poverty targeting using ML. *WBER* 32(3).
-  Mitchell, T. (1998). *Machine Learning*. McGraw-Hill.
-  Mitchell, M. et al. (2019). Model cards for model reporting. *FAT* 2019*.
-  Peng, R.D. (2011). Reproducible research in computational science. *Science* 334(6060).

-  Roberts, D.R. et al. (2017). CV strategies for data with structure. *Ecography* 40(8).
-  Saito, T., & Rehmsmeier, M. (2015). The PR plot is more informative than ROC under imbalance. *PLOS ONE* 10(3).
-  Vapnik, V. (1998). *Statistical Learning Theory*. Wiley.